

EXERCISE SOLUTIONS



Module 2: SQL Fundamentals (DDL & DML)

Module 2 - 8 Hours

SQL Fundamentals (DDL & DML)

Notes for the trainer

Core and stretch. The questions marked **(Core)** are the required set that fits the session. Debrief those with the whole room. Cover the unmarked stretch questions only if the batch worked ahead, or leave them as take-home reading.

Release after the practice window closes. Marks total 100.

Section	Questions	Marks
Level 1 - Understanding	Q1 to Q10	30
Level 2 - Applying SQL	Q11 to Q25	50
Level 3 - Design and judgement	Q26 to Q30	20

Expected outputs assume the standard seed data plus the 20 Smart Saver pilot bookings.

Level 1 - Understanding

Q1 (Core)

Item	Type	Reason
PAN number, exactly 10 characters	<code>CHAR(10)</code>	Genuinely fixed length, so padding is not a defect
Interest credited at maturity	<code>NUMERIC(15,2)</code>	Money must be exact
Tenure in days, max 3,650	<code>SMALLINT</code>	Fits comfortably; <code>INTEGER</code> wastes space on a large table
Moment a UPI payment was authorised	<code>TIMESTAMP</code>	An instant, not a calendar day
Loan instalment due date	<code>DATE</code>	A calendar day; the time is meaningless
Free-text complaint note	<code>TEXT</code> or <code>VARCHAR(4000)</code>	Unbounded prose
Counter of every transaction ever	<code>BIGINT</code>	<code>INTEGER</code> will overflow once history is migrated in
Auto-renew flag	<code>CHAR(1)</code> with a <code>CHECK</code> for 'Y'/'N', or <code>BOOLEAN</code>	Exactly two states

Accept `BOOLEAN` for the last item on PostgreSQL. Point out that Oracle had no `BOOLEAN` SQL type before 23c, which is why `CHAR(1)` is the portable banking convention.

Q2 (Core)

```
SELECT CAST(0.1 AS REAL) + CAST(0.2 AS REAL) = CAST(0.3 AS REAL);    -- false
SELECT CAST(0.1 AS NUMERIC) + CAST(0.2 AS NUMERIC) = CAST(0.3 AS NUMERIC); -- true
```

Business impact: each rounding error is invisible, but summed across two million transactions a month the branch cash position no longer balances, and the difference cannot be attributed to any individual transaction, so it cannot be corrected - only written off.

Q3

Constraint	Guarantees	ICICI Bank example
<code>NOT NULL</code>	A value must be supplied	<code>principal_amount</code> on a deposit
<code>UNIQUE</code>	No two rows share the value	<code>receipt_number</code> printed on the customer's receipt
<code>PRIMARY KEY</code>	Unique and not null; the row's identity	<code>deposit_id</code>
<code>FOREIGN KEY</code>	The value exists in the parent table	<code>fixed_deposit.customer_id</code> must be a real customer
<code>CHECK</code>	The value satisfies a condition	<code>principal_amount >= 10000</code> , the product minimum
<code>DEFAULT</code>	Supplies a value when the caller omits one	<code>status</code> defaults to 'ACTIVE'

Q4 (Core)

Two differences: a **PRIMARY KEY** cannot contain NULL and there is exactly one per table, whereas a **UNIQUE** constraint usually permits NULLs and a table may have several.

`deposit_id` was chosen because it is a **surrogate** key with no business meaning, so it never has to change. `receipt_number` is a **natural** key carrying business meaning, and business meaning changes: the bank could redesign the receipt format, merge with another bank's numbering, or discover a branch has been issuing duplicates. Every foreign key and every application reference would then have to change with it. Identity must be stable.

Q5 (Core)

	DELETE	TRUNCATE	DROP
Removes	Rows matching <code>WHERE</code>	All rows	The table definition and all rows
<code>WHERE</code> allowed	Yes	No	No
Rollback	Yes	PostgreSQL and SQL Server yes; Oracle and MySQL no	Rarely
Triggers fire	Yes	No	No
Speed on 10 million rows	Minutes, and it grows the transaction log	Immediate	Immediate

Q6

The job deletes transactions older than 90 days, but the regulator requires seven years. Within a quarter the bank would be unable to produce its own history, which is a compliance failure, not merely a data-loss bug. It also runs as a single enormous **DELETE**, which on a 40-million-row table will hold locks, bloat the transaction log and quite possibly time out.

What the bank should do instead: keep all seven years online, and manage size with **partitioning** by month so that queries touch only the partitions they need. Older partitions can be moved to cheaper storage or archived to a data warehouse, and only data beyond the seven-year retention period is removed, by dropping a whole partition rather than deleting rows. Partitioning is covered in Module 5, and the archival pipeline in Modules 10 and 11.

Q7 (Core)

Statement	Result	Why				
<code>SELECT NULL = NULL;</code>	NULL	Unknown compared to unknown is unknown, not true				
<code>SELECT NULL <> NULL;</code>	NULL	Same reason				
<code>SELECT 5000 + NULL;</code>	NULL	Any arithmetic with NULL is NULL				

Statement	Result	Why				
<code>SELECT COALESCE(NULL, NULL, 'FALLBACK');</code>	FALLBACK	Returns the first non-null argument				
<code>'SELECT 'A' \</code>	<code>\</code>	NULL <code>\</code>	<code>\</code>	<code>'</code>	NULL	Concatenation with NULL yields NULL in PostgreSQL and Oracle

The last one catches people out. Building an address line by concatenating fields where one is NULL produces a NULL address, not a partial one. Wrap each field in `COALESCE`.

Q8

`AND` has higher precedence than `OR`, so the first query is read as:

```
WHERE branch_id = 1 OR (branch_id = 2 AND principal_amount > 500000)
```

It returns **every** branch 1 deposit regardless of size, plus only the large branch 2 deposits. Against the pilot data that is 5 rows: FD2026000001, FD2026000009 and FD2026000017 from branch 1, plus FD2026000010 and FD2026000018 from branch 2.

The bracketed version restricts both branches to deposits above 500,000, returning 4 rows: FD2026000009, FD2026000010, FD2026000017 and FD2026000018. FD2026000001 at 25,000 drops out.

Q9 (Core)

`txn_date` is a `TIMESTAMP`. The literal `DATE '2026-03-31'` is converted to `2026-03-31 00:00:00`, so `BETWEEN` covers only the first instant of 31 March. Every transaction that happened during the working day on 31 March fails the test, and the same is true at the end of every quarter the report has ever run.

```
WHERE txn_date >= DATE '2026-01-01'
      AND txn_date <  DATE '2026-04-01'
```

This half-open range includes the whole of 31 March and excludes 1 April, and it stays correct if the column later gains sub-second precision.

Q10 (Core)

`WHERE` is evaluated **before** `SELECT`, so at the time `WHERE` runs the alias does not yet exist. `ORDER BY` is evaluated **after** `SELECT`, so by then it does.

Evaluation order is `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, `LIMIT`. This one fact explains several rules that otherwise look arbitrary, including why aggregate filters go in `HAVING` rather than `WHERE`, which you meet in Module 3.

Level 2 - Applying SQL

Q11 (Core)

```
CREATE TABLE recurring_deposit (
  rd_id          INTEGER      NOT NULL,
  customer_id    INTEGER      NOT NULL,
  branch_id      INTEGER      NOT NULL,
  rd_number      VARCHAR(20)  NOT NULL,
  monthly_amount NUMERIC(15,2) NOT NULL,
  interest_rate  NUMERIC(5,2) NOT NULL,
  tenure_months  SMALLINT     NOT NULL,
  start_date     DATE          DEFAULT CURRENT_DATE NOT NULL,
  maturity_date  DATE          NOT NULL,
  installments_paid INTEGER     DEFAULT 0 NOT NULL,
  status         VARCHAR(12)   DEFAULT 'ACTIVE' NOT NULL,
  CONSTRAINT pk_recurring_deposit PRIMARY KEY (rd_id),
  CONSTRAINT uq_rd_number         UNIQUE (rd_number),
  CONSTRAINT fk_rd_customer       FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
  CONSTRAINT fk_rd_branch        FOREIGN KEY (branch_id)   REFERENCES branch(branch_id),
  CONSTRAINT ck_rd_monthly       CHECK (monthly_amount >= 500),
  CONSTRAINT ck_rd_rate          CHECK (interest_rate > 0 AND interest_rate <= 12),
  CONSTRAINT ck_rd_tenure        CHECK (tenure_months BETWEEN 6 AND 120),
  CONSTRAINT ck_rd_maturity      CHECK (maturity_date > start_date),
  CONSTRAINT ck_rd_installments  CHECK (installments_paid >= 0),
  CONSTRAINT ck_rd_status        CHECK (status IN ('ACTIVE', 'MATURED', 'DEFAULTED', 'CLOSED'))
);
```

Marking: 1 mark per correctly typed column group, 1 per constraint, 2 for naming every constraint. Deduct if `installments_paid` is `NOT NULL` without a `DEFAULT`, which would force every insert to supply a zero.

Q12 (Core)

#	Statement fragment	Expected error
1	<code>monthly_amount 100</code>	violates check constraint "ck_rd_monthly"
2	<code>customer_id 5555</code>	violates foreign key constraint "fk_rd_customer" - Key (customer_id)=(5555) is not present in table "customer"
3	<code>maturity before start</code>	violates check constraint "ck_rd_maturity"
4	<code>duplicate rd_number</code>	duplicate key value violates unique constraint "uq_rd_number"
5	<code>status = 'PAUSED'</code>	violates check constraint "ck_rd_status"
6	<code>installments_paid = -3</code>	violates check constraint "ck_rd_installments"

Example for case 3:

```
INSERT INTO recurring_deposit
  (rd_id, customer_id, branch_id, rd_number, monthly_amount, interest_rate,
   tenure_months, start_date, maturity_date)
VALUES (8099, 1001, 1, 'RDTEST99', 5000, 7.10, 24,
       DATE '2026-03-01', DATE '2025-03-01');
```

The point of the exercise: every message names the rule that was broken. That is only true because the constraints were named in Q11.

Q13

```
INSERT INTO recurring_deposit
(rd_id, customer_id, branch_id, rd_number, monthly_amount,
 interest_rate, tenure_months, start_date, maturity_date)
VALUES
(8001, 1001, 1, 'RD2026000001', 5000.00, 7.10, 24, DATE '2026-03-01', DATE '2028-03-01'),
(8002, 1005, 5, 'RD2026000002', 2500.00, 6.85, 12, DATE '2026-03-05', DATE '2027-03-05'),
(8003, 1012, 4, 'RD2026000003', 10000.00, 7.35, 60, DATE '2026-03-10', DATE '2031-03-10');
```

Expected: `INSERT 0 3`. `installments_paid` and `status` were omitted and take their defaults of 0 and 'ACTIVE'.

Q14

```
ALTER TABLE recurring_deposit ADD COLUMN nominee_name VARCHAR(60);

UPDATE recurring_deposit
SET    nominee_name = 'Meera Reddy'
WHERE  rd_id = 8001;

SELECT rd_id, rd_number, nominee_name FROM recurring_deposit ORDER BY rd_id;
```

rd_id	rd_number	nominee_name
8001	RD2026000001	Meera Reddy
8002	RD2026000002	
8003	RD2026000003	

The blanks are NULL, not empty strings. The column had to be nullable, because nomination is optional and two rows already existed when it was added.

Q15

Check first:

```
SELECT rd_id, rd_number, monthly_amount
FROM   recurring_deposit
WHERE  monthly_amount < 1000;
```

Zero rows, so the change is safe. Now apply it:

```
ALTER TABLE recurring_deposit DROP CONSTRAINT ck_rd_monthly;
ALTER TABLE recurring_deposit
ADD CONSTRAINT ck_rd_monthly CHECK (monthly_amount >= 1000);
```

If any row had violated the new rule, the `ADD CONSTRAINT` would have failed with `check constraint "ck_rd_monthly" is violated by some row`. **Adding a constraint validates the data already present**, which makes it a free audit of historical data - and a reason to run the `SELECT` first, so you find out on your

terms.

Q16

```
ALTER TABLE recurring_deposit ADD COLUMN scheme_code VARCHAR(8) NOT NULL;
```

```
ERROR: column "scheme_code" of relation "recurring_deposit" contains null values
```

The three existing rows would have no value, and the column forbids NULL. Two ways to fix it:

```
-- Option A: give existing rows a default
ALTER TABLE recurring_deposit
  ADD COLUMN scheme_code VARCHAR(8) NOT NULL DEFAULT 'RDSTD';

-- Option B: add it nullable, populate, then tighten
ALTER TABLE recurring_deposit ADD COLUMN scheme_code VARCHAR(8);
UPDATE recurring_deposit SET scheme_code = 'RDSTD';
ALTER TABLE recurring_deposit ALTER COLUMN scheme_code SET NOT NULL;
```

Option B is what you use in production on a large table when the correct value differs per row. Option A is fine when one value genuinely applies to all history.

Q17 (Core)

```
SELECT receipt_number, principal_amount, interest_rate, maturity_date
FROM   fixed_deposit
WHERE  principal_amount >= 500000
ORDER BY principal_amount DESC;
```

receipt_number	principal_amount	interest_rate	maturity_date
FD2026000010	250000.00	7.40	2028-03-09
FD2026000020	250000.00	6.90	2026-09-16
FD2026000009	150000.00	7.50	2027-03-09
FD2026000019	150000.00	6.50	2026-06-17
FD2026000008	100000.00	6.90	2026-09-04
FD2026000018	100000.00	7.75	2031-03-17
FD2026000007	75000.00	6.50	2026-06-05
FD2026000017	75000.00	7.85	2029-03-16
FD2026000006	50000.00	7.75	2031-03-05
FD2026000016	50000.00	7.40	2028-03-15

Ten rows. Note the ties: five pairs share a principal, and their relative order is **not defined** by this query. Add a second sort key such as `receipt_number` if a stable order matters, which it does for anything a person will compare between runs.

Q18

```
SELECT receipt_number, booked_date, principal_amount
FROM   fixed_deposit
WHERE  booked_date >= DATE '2026-03-01'
      AND booked_date <= DATE '2026-03-10'
```

```
ORDER BY booked_date, principal_amount DESC;
```

Ten rows, FD2026000001 through FD2026000010, one per day. `BETWEEN` is safe here because `booked_date` is a `DATE` column with no time component. On `account_txn.txn_date`, which is a `TIMESTAMP`, it would not be.

Q19 (Core)

```
-- Deposits with no nominee
SELECT receipt_number, nominee_name
FROM   fixed_deposit
WHERE  nominee_name IS NULL
ORDER BY receipt_number;
```

Six rows: FD2026000003, 000006, 000009, 000012, 000015 and 000018.

```
-- Nominee is not Priya Verma, including those with no nominee
SELECT receipt_number, COALESCE(nominee_name, '(none)') AS nominee
FROM   fixed_deposit
WHERE  nominee_name <> 'Priya Verma' OR nominee_name IS NULL
ORDER BY receipt_number;
```

Why `<>` alone is not enough: for a row where `nominee_name` is `NULL`, the expression `nominee_name <> 'Priya Verma'` evaluates to `NULL`, not `TRUE`. `WHERE` keeps only rows that evaluate to `TRUE`, so those six rows are silently dropped. In banking terms, a "customers who did not choose Priya Verma" list would omit every customer who nominated nobody at all - exactly the group most worth calling.

Q20 (Core)

```
SELECT receipt_number,
       principal_amount,
       ROUND(principal_amount * interest_rate / 100, 2) AS annual_interest
FROM   fixed_deposit
ORDER BY annual_interest DESC
LIMIT 5;
```

receipt_number	principal_amount	annual_interest
FD2026000010	2500000.00	185000.00
FD2026000020	2500000.00	172500.00
FD2026000009	1500000.00	112500.00
FD2026000019	1500000.00	97500.00
FD2026000018	1000000.00	77500.00

Sorting by the alias `annual_interest` is legal because `ORDER BY` is evaluated after `SELECT`. Putting `annual_interest > 50000` in `WHERE` would fail for the same reason, in reverse.

Q21

```
SELECT fd.receipt_number,
       c.first_name || ' ' || c.last_name AS customer_name,
       fd.principal_amount,
       CASE WHEN fd.principal_amount >= 1000000 THEN 'HIGH VALUE'
```

```

        WHEN fd.principal_amount >= 100000 THEN 'STANDARD'
        ELSE 'SMALL'
    END
    AS segment
FROM   fixed_deposit fd
JOIN   customer c ON c.customer_id = fd.customer_id
ORDER BY fd.principal_amount DESC;

```

CASE is evaluated top to bottom and stops at the first match, so the 1,000,000 test must come first. Reversing the two WHEN branches would label every high-value deposit as STANDARD, because 2,500,000 also satisfies `>= 100000`. This ordering mistake is common and silent.

Six deposits are HIGH VALUE, ten are STANDARD and four are SMALL.

Q22

```

-- PostgreSQL and MySQL
SELECT txn_id, account_id, amount, narration
FROM   account_txn
WHERE  narration LIKE 'UPI/%'
      AND amount > 50000
ORDER BY amount DESC
LIMIT 10;

-- SQL Server
SELECT TOP 10 txn_id, account_id, amount, narration
FROM   account_txn
WHERE  narration LIKE 'UPI/%' AND amount > 50000
ORDER BY amount DESC;

-- Oracle 12c and later
SELECT txn_id, account_id, amount, narration
FROM   account_txn
WHERE  narration LIKE 'UPI/%' AND amount > 50000
ORDER BY amount DESC
FETCH FIRST 10 ROWS ONLY;

```

Accept `channel = 'UPI'` instead of the `LIKE` on narration, and give a bonus mark for noticing it: `channel` is an indexed, constrained column, while `narration` is free text. Filtering on the column that carries the meaning is both faster and more reliable. `LIKE 'UPI/%'` is anchored at the start so it can still use an index; `LIKE '%UPI%'` could not.

Q23

```

BEGIN;

SELECT deposit_id, receipt_number, status, auto_renew
FROM   fixed_deposit WHERE deposit_id = 7009;

UPDATE fixed_deposit
SET    status = 'BROKEN', auto_renew = 'N'
WHERE  deposit_id = 7009;

SELECT deposit_id, receipt_number, status, auto_renew
FROM   fixed_deposit WHERE deposit_id = 7009;

```

```
ROLLBACK;
```

Before: FD2026000009 | ACTIVE | Y. After the update: FD2026000009 | BROKEN | N. After the rollback: back to ACTIVE | Y.

Marking: full marks only if the `SELECT` comes first and the same `WHERE` clause is used in both statements.

Q24 (Core)

```
SELECT deposit_id, receipt_number, maturity_date
FROM   fixed_deposit
WHERE  maturity_date <= DATE '2026-06-30' AND status = 'ACTIVE';
```

deposit_id	receipt_number	maturity_date
7001	FD2026000001	2026-05-30
7007	FD2026000007	2026-06-05
7013	FD2026000013	2026-06-11
7019	FD2026000019	2026-06-17

Four rows.

```
BEGIN;
UPDATE fixed_deposit
SET    status = 'MATURED'
WHERE  maturity_date <= DATE '2026-06-30' AND status = 'ACTIVE';
-- UPDATE 4
ROLLBACK;
```

`UPDATE 4` matches the four rows the `SELECT` returned. **That match is the check.** If the `UPDATE` had reported 18, the `status = 'ACTIVE'` condition was dropped, and 14 deposits that were already matured, broken or closed would have been silently reopened.

Q25

```
UPDATE recurring_deposit
SET    status = 'DEFAULTED'
WHERE  rd_id = 8002;
```

`installments_paid` is deliberately untouched: it records what the customer actually paid, and that fact does not change because the account defaulted. Overwriting it would destroy the evidence needed to calculate the refund.

```
-- Closing it, the banking way
UPDATE recurring_deposit SET status = 'CLOSED' WHERE rd_id = 8002;

-- Rather than
-- DELETE FROM recurring_deposit WHERE rd_id = 8002;
```

Why: the regulator requires the bank to reproduce its own history, and the customer may dispute the default years later. A closed row still answers those questions; a deleted row cannot. Deleting also breaks any downstream report that already counted this deposit.

Level 3 - Design and judgement

Q26

Model answer.

Storing the amount as text is not acceptable. A `VARCHAR` amount column cannot be summed, cannot be compared numerically, cannot carry a `CHECK` constraint for a minimum booking, and will eventually be parsed differently by two different consumers - at which point the bank's own systems disagree about how much money a customer deposited. "Parse it when needed" means parsing it many times, in many places, inconsistently.

What I would insist on: `principal_amount NUMERIC(15,2) NOT NULL` in the ICICI Bank table, with a `CHECK` for the product minimum, and a documented contract with the vendor specifying a plain decimal number with a dot separator and no currency suffix or grouping.

Where the cleaning goes: in a **staging layer**, not in the target table. The feed lands in a staging table where the column is text exactly as supplied. A transformation step parses, validates and converts, writing clean rows into `fixed_deposit` and rejected rows into an error table with the reason. That way a malformed value is visible and reportable rather than silently coerced to zero, and the raw feed is retained for the vendor dispute that will follow. This is the extract-transform-load pattern from Module 11, and it is the standard answer to any untrusted feed.

Q27 (Core)

Model answer. `RESTRICT` is correct here.

A bank does not delete a customer. When a relationship ends, the customer record is closed and retained, because the regulator requires several years of history and because the customer may return or dispute something later. If `CASCADE` were in place, one mistaken `DELETE` against `customer` would silently remove that customer's deposits, and with cascades elsewhere in the schema, potentially their accounts and transactions too - with no error and nothing in the log to say what vanished.

`RESTRICT` turns that mistake into an error message, which is exactly what you want: the delete fails, nothing is lost, and someone investigates. The tidying-up argument is real but is solved differently, by an explicit archival process that closes the deposits deliberately, in a known order, inside a transaction, with a record of what it did.

Accept the opposite position only if the learner restricts `CASCADE` to a genuinely dependent child such as a junction or preference table, and explicitly excludes financial records.

Q28

Model answer.

```
ALTER TABLE account_txn ADD COLUMN authorised_by INTEGER NULL;

ALTER TABLE account_txn
  ADD CONSTRAINT fk_txn_authorised_by
    FOREIGN KEY (authorised_by) REFERENCES employee(employee_id);
```

Decision	Choice	Reason
Type	INTEGER	Must match <code>employee.employee_id</code> exactly or the foreign key and any index on it are useless
Nullability	Nullable	Most transactions are ATM, UPI and POS, with no employee involved. "Where applicable" in the requirement means the fact is genuinely absent, which is what NULL is for
Foreign key	Yes	An authorisation attributed to a non-existent employee is worthless in an audit
Existing 40 million rows	Leave them NULL	They were never authorised by an employee; inventing a value would be fabricating audit data

On the 40 million existing rows, the important operational point is that adding a **nullable column with no default** is a metadata-only change on modern PostgreSQL, Oracle and SQL Server, so it completes almost instantly. Adding a **NOT NULL** column with a default historically rewrote the entire table, locking it for the duration - on 40 million rows in a core banking system that is an outage. Always check which behaviour your engine and version has before altering a large table.

Bonus mark for proposing an index on `authorised_by` only if the audit team will query by employee, and for noting that an unused index still costs write time on every insert.

Q29 (Core)

Model answer.

Approach	Advantages
CHECK constraint	Simpler: no join needed to read the status, no extra table to maintain. Faster: the value is in the row, so reports do not join. The rule is visible in the table definition itself.
Lookup table with a foreign key	Adding or renaming a status is a data change rather than a DDL change, so no ALTER TABLE and no downtime. The lookup can carry extra attributes such as a display label, a sort order, a description, or whether the status is terminal.

Choice for this table: keep the CHECK constraint. The four values are defined by the product design and change roughly never; they carry no extra attributes; and every report that reads a deposit would otherwise pay for a join to discover a four-character word it already had. The cost of change - one **ALTER TABLE** in the rare event a fifth status appears - is lower than the cost of the join and the extra table.

The decision flips when any of these becomes true: the list grows past roughly a dozen values, non-technical users need to manage it, the values need attributes of their own, or a **DEPLOY** requiring **ALTER TABLE** is operationally expensive in that environment.

Q30

Model answer.

Immediate actions, in order:

1. Do not run anything else on that connection, and do not let anyone try to "fix it" with another **UPDATE**. Stop the bleeding first.
2. Establish the blast radius: **SELECT status, COUNT(*) FROM account GROUP BY status;** tells you how many rows now read DORMANT. It is 45 in this database and would be every account in production.

3. Check whether the previous values are recoverable. Options, best first: a point-in-time restore to just before the statement, a read replica or standby that has not yet applied it, a flashback query if the engine supports it, or last night's backup plus the transaction log.
4. Raise it as an incident immediately and tell the application teams, because downstream systems may already have consumed the wrong statuses.
5. Restore, verify the counts against a known-good reference, and only then let normal traffic resume.

Two working-practice changes:

1. **Nobody connects to production with an account that can run unqualified DML.** Day-to-day work uses a read-only role; changes go through a reviewed, scripted release with a named approver. This removes the whole class of accident rather than asking people to be careful.
2. **Make the safe path the easy path.** Standardise on a client configured with autocommit off, so an accidental statement can be rolled back, and adopt the team habit of writing the `SELECT` with the intended `WHERE` clause first and converting it to the `UPDATE` only after checking the row count. Where the tool supports it, enable a safe-update mode that refuses DML with no `WHERE` clause.

Marking: 5 marks. Look for stopping before fixing, assessing scope, a real recovery route, raising an incident, and two practice changes that are systemic rather than "be more careful". Deduct for any answer that blames the individual.